

REVERSE ENGINEERING // MASTERY COURSE

PHASE 7

Mobile RE

Android (APK, smali) aur iOS (IPA) apps reverse karna.

```
re@learning: ~/phase-7
$ ./start_learning.sh --phase 7
# Hinglish mein, simple examples ke saath - ek-ek step
$ status: READY ✓
```

 Detailed Theory

 Code + Line-by-line

 Interview Q&A

IN Hinglish

Reverse Engineering Mastery 34 • Self-Study Roadmap 6 • Phase 7 Handbook

Phase 7 — Mobile Reverse Engineering (Android + iOS)

Goal: Mobile apps (Android APK, iOS IPA) ko reverse karna seekhna. Ye Phase 0-6 ke concepts (assembly, static/dynamic, debugging) use karta hai, bas platform alag hai. Time: **6-8 hafte** (dono platforms). Base strong hoga toh aasan lagega.

7.0 — Mobile RE Kyun Alag Hai?

Desktop `.exe` aur mobile apps mein bada farak hai: - **Android apps** Java/Kotlin mein banti hain aur ek special "bytecode" (DEX) mein chalti hain — jo reverse karna **aasan** hai (bytecode → aasani se wapas Java jaisa). - **iOS apps** Swift/Objective-C mein banti hain aur seedha **native ARM machine code** mein — jo desktop RE jaisa **mushkil** hai.

Isliye tools aur approach dono alag hain. Ye phase do parts mein hai: - **Part A — Android RE** (pehle karo — free, aasan) - **Part B — iOS RE** (baad mein — Mac + jailbroken device chahiye)

PART A — ANDROID REVERSE ENGINEERING

 **Time: 4-5 hafte**

Android se shuru karo — free hai, tools easily milte hain, normal Windows/Linux PC pe ho jaata hai.

7A.1 — Android Basics (Neev)

APK file kya hai?

APK = Android Package. Ye asal mein ek **ZIP file** hai! `app.apk` ko `app.zip` rename karke khol lo — andar sab files dikh jaayengi.

APK ke andar kya hota hai

File/Folder	Kya Hai
<code>AndroidManifest.xml</code>	App ka "identity card" — permissions, components, entry point
<code>classes.dex</code>	★ Asli code (Dalvik bytecode) — yahin logic chhupa hai
<code>resources.arsc</code>	Compiled resources (strings, layouts)
<code>res/</code>	Images, layouts, icons
<code>lib/</code>	Native code (<code>.so</code> files — C/C++ compiled, ARM assembly)
<code>assets/</code>	Extra files (fonts, DBs, config)
<code>META-INF/</code>	Signature (app kis ne sign kiya)

App kaise banti hai (samajhna zaroori)

```
Java/Kotlin Code
  ↓ (compile)
.class (Java bytecode)
  ↓ (dx / d8 tool)
.dex (Dalvik bytecode)  <- RE mein hum ISKO wapas padhte hain
  ↓ (package + sign)
APK
```

DVM vs ART

Android apps ek Virtual Machine pe chalti hain — pehle **Dalvik VM (DVM)**, ab **ART (Android Runtime)**. Isliye code seedha "machine code" nahi, balki **bytecode** hota hai — jo desktop se reverse karna aasan hai!

Key point: DEX bytecode high-level info (class names, method names) rakhta hai, isliye reverse karke wapas Java jaisa code aasani se milta hai.

7A.2 — Smali Language (Android Ki "Assembly")

Ye Android RE ka **dil** hai. Jaise desktop mein Assembly thi, waise Android mein **Smali** hai.

Smali kya hai?

Smali, Dalvik bytecode ka human-readable version hai. `.dex` ko todoge toh smali dikhega. Ye Java se low-level hai but desktop assembly se aasan (registers aur methods clearly dikhte hain).

Example — Ek simple smali

```
.method public checkPassword(Ljava/lang/String;)Z
    .locals 2
    const-string v0, "admin123"          # v0 register mein "admin123" daalo
    invoke-virtual {p1, v0}, Ljava/lang/String;->equals(Ljava/lang/Object;)Z
    move-result v1                        # equals() ka result v1 mein
    return v1                             # true/false return
.end method
```

Line by line: - `.method public checkPassword(...)Z` — method ka naam, `Z` matlab boolean (true/false) return karega. - `const-string v0, "admin123"` — v0 register mein string "admin123" daali. - `invoke-virtual {p1, v0}, ...equals` — `p1` (user input) pe `.equals(v0)` call kiya — matlab input ko "admin123" se compare. - `move-result v1` — comparison ka result v1 mein. - `return v1` — result wapas.

Password mil gaya: `admin123` — smali padhna aa gaya toh aadha kaam ho gaya!

Smali seekhne ke key points

- **Registers:** `v0, v1...` (local) aur `p0, p1...` (parameters; `p0` = `this` object).
- **Common opcodes:** `const-string`, `invoke-virtual`, `invoke-static`, `if-eq`, `if-ne`, `move-result`, `return`, `goto`.
- **Types:** `Z` =boolean, `I` =int, `V` =void, `Ljava/lang/String;` =String object.

7A.3 — Android RE Tools

Tool	Kaam	Level
jadx / jadx-gui ★	APK ko seedha Java code mein dikhaata hai (best beginner tool)	Free
apktool ★	APK ko smali mein todta hai aur wapas build karta hai (patching)	Free
dex2jar	<code>.dex</code> ko <code>.jar</code> mein badalta hai	Free
JD-GUI / Bytecode Viewer	Java code dekhna	Free
Ghidra / IDA	<code>lib/*.so</code> (native C/C++) reverse karna	Free
Android Studio + Emulator	App chalana/debug karna	Free
Frida ★★	Runtime hooking (dynamic) — sabse powerful	Free

Beginner recommendation: **jadx-gui** se shuru karo. APK drag-drop karo, turant Java-jaisa code dikh jaayega.

7A.4 — Android Static Analysis

Bina app chalaye samajhna:

1. **APK ko jadx-gui mein kholo** → Java code padho.
2. **AndroidManifest.xml padho** → kaunsi Activity pehle chalti hai (entry point, usually `MainActivity`), kya permissions maangi.
3. **Strings dhoondo** → URLs, API keys, passwords aksar hardcoded milte hain.
4. **Interesting functions** → login, password check, encryption wale functions dhoondo.
5. **Native `.so` files** → agar logic native code mein hai toh Ghidra mein kholo (yahan Phase 1 ki ARM assembly kaam aayegi!).

Real example flow

```
Ek "Login" app mila
→ jadx-gui mein khola
→ MainActivity dhoondi
→ checkLogin() function mila
→ dekha: if(password.equals("Sup3rS3cr3t"))
→ Password mil gaya bina app chalaye! 🎉
```

7A.5 — Android Dynamic Analysis

App ko **chale hue** dekhna: - **Emulator setup** — Android Studio ka emulator (safe, computer pe hi Android chalega). Rooted emulator best hai. - **Logcat** — app ke logs dekhna (developers debug info chhod dete hain). - **Frida** ★★ — Sabse powerful tool. Chalti hui app mein "hook" laga ke functions ko rok/badal sakte ho, runtime values dekh sakte ho. - Example: `equals()` function hook karke dekho asli value kya compare ho rahi. - **Objection** — Frida ke upar bana, aasan commands (SSL pinning bypass, root detection bypass). - **Burp Suite / mitmproxy** — app ka network traffic dekhna (kaunse server se baat kar rahi).

Frida hooking example (concept)

```
// login function ko hook karke uske arguments print karna
Java.perform(function() {
    var Login = Java.use("com.app.LoginActivity");
    Login.checkPassword.implementation = function(pass) {
        console.log("Password argument: " + pass); // asli input print
        return true; // hamesha true return - bypass!
    };
});
```

Samjho: - `Java.use(...)` — target class pakdi. - `.implementation = function(...)` — us method ko replace kiya. - `console.log(pass)` — jo bhi password aaya, print kar diya. - `return true` — function ko hamesha "sahi" bana diya (login bypass).

7A.6 — APK Patching (Modify & Rebuild)

App ka behaviour badalna:

```
apktool d app.apk          # 1. APK ko smali mein todo (decompile)
# 2. smali file edit karo (jaise "if-eqz" ko badlo ya return value change karo)
apktool b app_folder -o new.apk # 3. wapas APK banao (build)
# 4. APK ko sign karo (warna install nahi hoga)
apksigner sign --ks my.keystore new.apk
# 5. install & test (emulator/phone pe)
adb install new.apk
```

Example: Ek "premium locked" feature ko smali mein `return true` bana ke unlock karna (learning ke liye). Ya login check ko hamesha pass karwana.

⚠️ Sirf apni ya practice apps pe. Kisi ki paid app ki protection todna illegal hai.

7A.7 — Android Advanced Topics

- **Obfuscation** — ProGuard/R8/DexGuard se naam ulte (`a.a.a()`) — inhe trace karna.
- **Root detection bypass** — apps jo rooted device pe chalne se mana karti hain.
- **SSL Pinning bypass** — network traffic dekhne ke liye (Objection/Frida se).
- **Anti-tampering bypass** — apps jo modify hona detect karti hain.
- **Native code RE** — `.so` files (ARM assembly) — desktop RE skills seedha kaam aati hain.
- **Frida scripting** — apne custom hooks likhna.

✅ **Android Part Complete Jab:** Tum ek APK ko jadx mein khol ke logic samajh sako, smali padh sako, Frida se ek function hook kar sako, aur ek simple patch karke APK rebuild kar sako.



PART B — iOS REVERSE ENGINEERING

🕒 **Time: 3-4 hafte**

⚠️ **Note:** iOS RE thoda mushkil aur mehanga hai. Ideally chahiye: ek **Mac** (kuch tools sirf Mac pe) aur ek **jailbroken iPhone** (dynamic analysis ke liye). Bina inke bhi static analysis ho sakta hai, but limited. Isliye Android pehle solid karo.

iOS apps **Swift/Objective-C** mein banti hain aur seedha **ARM machine code** (native) mein compile hoti hain — Android ke bytecode jaisa aasan nahi. Yahan desktop RE skills (assembly, Ghidra) zyada kaam aati hain.

7B.1 — iOS App Basics

IPA file kya hai?

Android ke APK ki tarah, iOS ka app package. Ye bhi ek **ZIP** hai! Andar `Payload/AppName.app/` folder hota hai.

IPA ke andar kya hota hai

File	Kya Hai
<code>Payload/AppName.app/</code>	Asli app folder
<code>AppName</code> (binary)	★ Main executable — Mach-O format (ARM machine code)
<code>Info.plist</code>	App ki config/identity (Android ke Manifest jaisa)
<code>Frameworks/</code>	Extra libraries
<code>_CodeSignature/</code>	Apple ka digital signature
Assets (<code>.car</code>)	Images, resources

Mach-O format

iOS/Mac binaries ka format (jaise Windows mein PE, Linux mein ELF). Ghidra/IDA ise samajhte hain.

Encryption problem (★ important)

App Store se aane wali apps **encrypted** hoti hain (FairPlay DRM). Reverse karne ke liye pehle **decrypt** karni padti hai — jailbroken device pe, jab app memory mein decrypt ho jaati hai tab use dump karke (jaise `frida-ios-dump` se).

7B.2 — Objective-C & Swift Samjho

- **Objective-C** — purani but common. Ismein method calls ek special function `objc_msgSend` ke through hote hain — ye RE mein bahut dikhega.
- **Swift** — modern, but reverse karna thoda mushkil (name mangling — naam encode ho jaate hain).
- Basic samajh honi chahiye ki inka code kaisa dikhta hai — classes, methods, message passing.

7B.3 — iOS RE Tools

Tool	Kaam	Note
Ghidra / IDA Pro ★	Mach-O binary disassemble/decompile	Free/Paid
Hopper Disassembler	iOS/Mac ke liye popular, achha UI	Paid (trial)
class-dump	Objective-C classes/methods ki list nikaalna	Free
Frida ★	Dynamic analysis, hooking (jailbroken device)	Free
Objection	Frida-based, aasan commands	Free
frida-ios-dump	Encrypted app ko decrypt/dump karna	Free
otool / nm	Binary info, symbols dekhna (Mac tools)	Free

7B.4 — iOS Static Analysis

1. **IPA ko unzip karo** → binary aur `Info.plist` nikaalo.
2. **Info.plist padho** → permissions, URL schemes, config.
3. **App decrypt karo** (agar App Store se) → `frida-ios-dump` se.
4. **class-dump** → saari classes aur methods ki list (kya-kya functions hain).
5. **Ghidra/Hopper mein binary kholo** → interesting functions (login, crypto) dhoondo, ARM assembly padho.
6. **Strings** → hardcoded secrets, URLs.

7B.5 — iOS Dynamic Analysis

Iske liye **jailbroken iPhone** chahiye (ya kuch cases mein simulator, but limited). - **Frida hooking** — chalti hui app ke functions rok/badal ke dekhna (Android jaisa hi, but Objective-C/Swift methods). - **Objection** — SSL pinning bypass, jailbreak detection bypass, method hooking aasan commands se. - **Debugserver / LLDB** — Apple ka debugger, breakpoints lagana. - **Network analysis** — Burp Suite se traffic dekhna. - **Cycript / Frida REPL** — live app mein objects explore karna.

7B.6 — iOS Advanced Topics

- **FairPlay decryption** deeply samajhna.
- **Anti-jailbreak detection bypass** — apps jo jailbroken device pe chalne se mana karti hain.
- **SSL pinning bypass** — network dekhne ke liye.
- **Swift-specific RE** — name demangling, metadata.
- **Kernel-level / system RE** — bahut advanced.

✅ **iOS Part Complete Jab:** Tum ek IPA ko unzip/analyze kar sako, class-dump se structure samajh sako, Ghidra/Hopper mein binary explore kar sako, aur (jailbroken device ho toh) Frida se hook laga sako.

🔑 Android vs iOS — Quick Comparison

Cheez	Android	iOS
Package	<code>.apk</code> (ZIP)	<code>.ipa</code> (ZIP)
Code type	Bytecode (Dalvik/DEX) — aasan	Native ARM machine code — mushkil
Language	Java / Kotlin	Swift / Objective-C
"Assembly"	Smali	ARM Assembly
Main tool	jadx, apktool, Frida	Ghidra/Hopper, Frida
Encryption	Usually nahi	Haan (FairPlay DRM)
Setup cost	Free (koi PC)	Mehnga (Mac + jailbroken iPhone)
Difficulty	🟢 Aasan (shuru yahin se)	🔴 Mushkil (baad mein)

Rule of thumb: Android pehle poora seekho (aasan + free), phir iOS. **Frida dono mein common hai** — isko achhe se seekhna dono jagah kaam aayega.

📱 Mobile RE — Practice & Resources

- 🌐 **OWASP MASTG (Mobile App Security Testing Guide)** — mobile RE/security ki bible, free.
- 🌐 **MASVS** — mobile security standards.
- 🧪 **DIVA / InsecureBankv2 / OWASP UnCrackable** — jaan-boojh ke kamzor banaayi gayi practice apps (Android + iOS dono).

- 📱 Apni khud ki banaayi hui simple app reverse karke shuru karo (safe + legal).
- 📺 YouTube: LiveOverflow (mobile), 8ksec, TheGrayArea.

🎯 Key Takeaways (Ek Line Mein)

1. **APK aur IPA dono ZIP files hain** — rename karke khol sakte ho.
2. Android code = **bytecode (DEX)** — aasan; iOS code = **native ARM** — mushkil.
3. **Smali** = Android ki assembly; padhna aa gaya toh aadha kaam.
4. Android tools: **jadx** (Java dekho), **apktool** (patch), **Frida** (hook).
5. iOS apps **encrypted** (FairPlay) — pehle decrypt/dump karni padti (frida-ios-dump).
6. iOS tools: **Ghidra/Hopper**, **class-dump**, **Frida**, **Objection**.
7. **Frida dono platforms mein** sabse powerful — zaroor seekho.
8. **Android pehle** (free), phir iOS (Mac + jailbroken device).

✅ **Phase 7 Complete Jab:** Tum Android APK aur (resources ho toh) iOS IPA dono ko static + dynamic dono tarike se analyze kar sako, aur ek practice app ka logic/secret nikaal sako. Congratulations — ab tum ek Mobile RE bhi ho! 🎉 🚀

Phase 7 — Interview Questions & Answers (Hinglish)

Mobile RE (Android + iOS) se related important interview questions, simple Hinglish jawaab ke saath.

Q1. APK file kya hoti hai?

Ans: APK (Android Package) ek Android app ka installation package hai. Ye asal mein ek **ZIP file** hoti hai — rename karke khol sakte ho. Andar `AndroidManifest.xml`, `classes.dex` (asli code), resources, `lib/` (native code), aur signature hoti hai.

Q2. `classes.dex` file mein kya hota hai?

Ans: `classes.dex` mein app ka asli code hota hai — Dalvik bytecode ke form mein. Ye wahi code hai jo Android ki virtual machine (ART/Dalvik) chalati hai. RE mein isi ko reverse karke wapas Java/smali mein padhte hain.

Q3. Smali kya hai?

Ans: Smali, Dalvik bytecode ka human-readable version hai — Android ki "assembly language". Jab `.dex` ko decompile karte hain toh smali milta hai. Ye Java se low-level hai but desktop assembly se aasan, kyunki ismein registers (`v0`, `p1`) aur method names clearly dikhte hain.

Q4. DVM/ART kya hai aur Android code reverse karna aasan kyun hota hai?

Ans: Android apps ek virtual machine pe chalti hain — pehle Dalvik VM (DVM), ab ART (Android Runtime). Isliye app ka code seedha machine code nahi, balki **bytecode** hota hai jo high-level info (class/method names) rakhta hai. Isliye reverse karke wapas Java-jaisa readable code aasani se mil jaata hai — desktop native code se kaafi aasan.

Q5. jadx aur apktool mein kya farak hai?

Ans: **jadx** APK ko seedha readable Java code mein dikhata hai — code samajhne (static analysis) ke liye best. **apktool** APK ko smali mein todta hai aur usse wapas APK build bhi kar sakta hai — matlab patching (code badalkar rebuild) ke liye use hota hai.

Q6. AndroidManifest.xml kyun important hai?

Ans: Ye app ka "identity card" hai. Ismein app ke saare components (Activities, Services), permissions (kya-kya access maangti hai), aur entry point (kaunsi Activity pehle chalti hai, usually MainActivity) hote hain. RE shuru karte waqt yahan se pata chalta hai app kahan se shuru hoti hai aur kya karti hai.

Q7. Frida kya hai aur mobile RE mein kyun important hai?

Ans: Frida ek dynamic instrumentation tool hai jo chalti hui app mein functions ko "hook" karne deta hai — matlab unhe rok sakte ho, arguments/return values dekh/badal sakte ho, ya poora replace kar sakte ho. Ye Android aur iOS **dono** mein kaam karta hai, isliye mobile RE ka sabse powerful aur zaroori tool hai (jaise login bypass, SSL pinning bypass).

Q8. APK patching ka process kya hai?

Ans: Steps: (1) `apktool d app.apk` se APK ko smali mein decompile karo, (2) smali code edit karo (jaise koi check ka result badal do), (3) `apktool b` se wapas APK build karo, (4) APK ko sign karo (`apksigner`), warna install nahi hoga, (5) `adb install` se emulator/phone pe install karke test karo.

Q9. IPA file kya hoti hai aur APK se kaise alag hai?

Ans: IPA iOS app ka package hai (APK ka iOS version), aur ye bhi ek ZIP file hai jisme `Payload/AppName.app/` folder hota hai. Bada farak: IPA ke andar binary **native ARM machine code** (Mach-O format) mein hoti hai — Android ke aasan bytecode se mushkil. Aur App Store apps **encrypted** (FairPlay) hoti hain.

Q10. iOS apps reverse karna Android se mushkil kyun hai?

Ans: Do wajah: (1) iOS apps native ARM machine code mein compile hoti hain (Android ki tarah high-level bytecode nahi), isliye readable code wapas nahi milta — assembly padhni padti hai. (2) App Store apps FairPlay DRM se encrypted hoti hain, jinhe pehle jailbroken device pe decrypt/dump karna padta hai. Plus setup mehnga hai (Mac + jailbroken iPhone).

Q11. FairPlay encryption kya hai aur ise kaise handle karte hain?

Ans: FairPlay Apple ka DRM hai jo App Store apps ki binary ko encrypt kar deta hai, taaki koi seedha reverse na kar sake. Handle karne ke liye jailbroken device pe app ko chalte hain — jab app memory mein khud ko decrypt kar leti hai, tab `frida-ios-dump` jaise tool se decrypted binary memory se dump kar lete hain, jise phir Ghidra/Hopper mein analyze karte hain.

Q12. class-dump tool kya karta hai?

Ans: class-dump iOS/Mac binary se Objective-C ki saari classes, methods, aur properties ki list nikaal deta hai (header files jaise). Isse ek hi jhalak mein pata chal jaata hai ki app mein kaunse functions hain — jaise `checkLogin`, `isPremium` — aur kahan analysis focus karni hai.

Q13. SSL pinning kya hai aur use bypass kyun karte hain?

Ans: SSL pinning ek security technique hai jismein app sirf ek specific certificate ko trust karti hai, taaki koi beech mein (proxy) traffic na dekh sake. RE/testing mein hum app ka network traffic dekhna chahte hain (Burp Suite se), isliye SSL pinning ko bypass karte hain (Objection/Frida se), taaki traffic proxy se guzre aur dikh sake.

Q14. Android aur iOS RE mein kya common cheez seekhna best hai?

Ans: Frida — kyunki ye dono platforms mein kaam karta hai. Ek baar Frida hooking achhe se seekh li toh Android aur iOS dono ki dynamic analysis (function hooking, bypass, runtime inspection) cover ho jaati hai. Isliye ye mobile RE ka sabse high-value skill hai.

Q15. Mobile app ki native `.so` files kya hoti hain aur unhe kaise reverse karte hain?

Ans: `.so` files Android APK ke `lib/` folder mein hoti hain — ye C/C++ mein likha, ARM assembly mein compiled native code hai (jaise performance ya security-sensitive logic). Inhe reverse karne ke liye Ghidra/IDA use karte hain — yahan desktop RE aur ARM assembly ki skills seedha kaam aati hain, kyunki ye bytecode nahi native code hai.

Phase 7 — Interview Questions & Answers (English)

The same Mobile RE (Android + iOS) interview questions, answered in clear English.

Q1. What is an APK file?

Ans: An APK (Android Package) is an Android app's installation package. It's actually a **ZIP file** — you can rename and open it. Inside are `AndroidManifest.xml`, `classes.dex` (the actual code), resources, `lib/` (native code), and the signature.

Q2. What is stored in the `classes.dex` file?

Ans: `classes.dex` contains the app's actual code as Dalvik bytecode — the code Android's virtual machine (ART/Dalvik) runs. In RE we reverse it back into Java/smali to read it.

Q3. What is Smali?

Ans: Smali is the human-readable form of Dalvik bytecode — Android's "assembly language." Decompiling a `.dex` yields smali. It's lower-level than Java but easier than desktop assembly, since it clearly shows registers (`v0`, `p1`) and method names.

Q4. What is DVM/ART, and why is Android code easier to reverse?

Ans: Android apps run on a virtual machine — formerly Dalvik VM (DVM), now ART (Android Runtime). So the app's code is **bytecode**, not raw machine code, and it retains high-level info (class/method names). This makes it easy to reverse back into readable Java-like code — far easier than desktop native code.

Q5. What is the difference between jadx and apktool?

Ans: **jadx** shows an APK directly as readable Java code — best for understanding code (static analysis). **apktool** breaks an APK into smali and can rebuild it into an APK — used for patching (modifying code and rebuilding).

Q6. Why is AndroidManifest.xml important?

Ans: It's the app's "identity card." It lists all components (Activities, Services), permissions (what access it requests), and the entry point (which Activity launches first, usually MainActivity). When starting RE, it tells you where the app begins and what it does.

Q7. What is Frida and why is it important in mobile RE?

Ans: Frida is a dynamic instrumentation tool that lets you "hook" functions in a running app — pause them, view/modify arguments and return values, or replace them entirely. It works on **both** Android and iOS, making it the most powerful and essential mobile RE tool (e.g., login bypass, SSL pinning bypass).

Q8. What is the APK patching process?

Ans: Steps: (1) `apktool d app.apk` to decompile the APK into smali, (2) edit the smali code (e.g., change a check's result), (3) `apktool b` to rebuild the APK, (4) sign the APK (`apksigner`), otherwise it won't install, (5) `adb install` to install and test on an emulator/phone.

Q9. What is an IPA file and how does it differ from an APK?

Ans: An IPA is an iOS app package (the iOS counterpart of an APK), and it's also a ZIP file containing a `Payload/AppName.app/` folder. Key difference: the IPA's binary is **native ARM machine code** (Mach-O format) — harder than Android's bytecode. Also, App Store apps are **encrypted** (FairPlay).

Q10. Why is reversing iOS apps harder than Android?

Ans: Two reasons: (1) iOS apps compile to native ARM machine code (not high-level bytecode like Android), so you don't get readable code back — you must read assembly. (2) App Store apps are encrypted with FairPlay DRM and must first be decrypted/dumped on a jailbroken device. Plus the setup is expensive (Mac + jailbroken iPhone).

Q11. What is FairPlay encryption and how do you handle it?

Ans: FairPlay is Apple's DRM that encrypts App Store app binaries so they can't be directly reversed. To handle it, you run the app on a jailbroken device — once the app decrypts itself in memory, you dump the decrypted binary from memory using a tool like `frida-ios-dump`, then analyze it in Ghidra/Hopper.

Q12. What does the class-dump tool do?

Ans: class-dump extracts all Objective-C classes, methods, and properties from an iOS/Mac binary (like header files). At a glance it reveals which functions exist — such as `checkLogin`, `isPremium` — and where to focus your analysis.

Q13. What is SSL pinning and why do you bypass it?

Ans: SSL pinning is a security technique where an app trusts only a specific certificate, so no one in the middle (a proxy) can read its traffic. In RE/testing we want to see the app's network traffic (via Burp Suite), so we bypass SSL pinning (using Objection/Frida) to let the traffic pass through the proxy and be visible.

Q14. What common skill is best to learn for both Android and iOS RE?

Ans: Frida — because it works on both platforms. Once you learn Frida hooking well, you cover dynamic analysis (function hooking, bypasses, runtime inspection) for both Android and iOS. It's the highest-value skill in mobile RE.

Q15. What are a mobile app's native `.so` files and how do you reverse them?

Ans: `.so` files live in an Android APK's `lib/` folder — they're native code written in C/C++ and compiled to ARM assembly (for performance- or security-sensitive logic). You reverse them with Ghidra/IDA — here your desktop RE and ARM assembly skills apply directly, since this is native code, not bytecode.